

Documentation Jour 3 - Évolution de Sécurité & Dashboard 3D



1. Objectif de l'Étape 3

Projet : Physical AI - ABA Fusion | **Auteur :** Hafida Belayd



Nouveautés du Jour 3 : Cette journée s'est concentrée sur la fiabilisation du robot physique avec l'ajout de mécanismes d'arrêt d'urgence (Safety Stop) matériels, l'amélioration du traitement des données asynchrones (Non-Blocking), et la consolidation de l'intégration avec le Dashboard 3D React.

Points Clés Abordés

- Implémentation d'un arrêt d'urgence matériel conditionné par la détection du capteur ultrasonique.
- Optimisation drastique de la latence (réduction des délais bloquants et augmentation de la fréquence de télémétrie).
- Ajout de mécanismes de "Failsafe" avancés (Watchdog de commandes et détection de perte réseau).

- Alignement esthétique de la documentation avec le thème Cyber-Industriel du Dashboard.
- Traçabilité améliorée du trafic dans la zone critique (Zone X).

2. Intégration du Capteur HC-SR04 et Système d'Arrêt Double (Dual Safety)

Le robot a été équipé d'un capteur à ultrasons **HC-SR04** pour détecter les obstacles frontaux. Pour garantir une sécurité absolue, la logique d'arrêt d'urgence a été implémentée à deux niveaux : matériel (C++) et logiciel (Python).

2.1. Niveau Matériel : C++ (ESP32 / Arduino)

Afin d'éviter toute collision physique en cas de latence réseau, un mécanisme autonome d'arrêt d'urgence est exécuté directement sur le microcontrôleur.

```
const float SAFETY_STOP_CM = 20.0;

bool isObstacleTooClose() {
    return latestDistance > 0.0 && latestDistance <= SAFETY_STOP_CM;
}

void applySafetyStop() {
    if (!currentlyClose) return;

    // Force les moteurs à 0 indépendamment des commandes réseau
    if (currentM1 > 0) setMotor(1, 0);
    if (currentM2 > 0) setMotor(2, 0);
}
```

Fiabilisation du HC-SR04 : Le code C++ utilise une boucle d'échantillonnage avec un timeout réduit à **10000 µs** (couvrant 1.7 mètres) pour éviter de bloquer le processeur si aucun écho n'est reçu. Si un obstacle apparaît à moins de 20cm, les moteurs sont coupés instantanément.

2.2. Niveau Logiciel : Jumeau Numérique (Python)

Les données du capteur HC-SR04 sont envoyées via télémetrie au simulateur Python. Le jumeau numérique analyse cette distance pour mettre à jour son propre état et synchroniser l'affichage 3D.

```
# Dans agv_agent.py
if self.is_digital_twin_active and (0 < self.distance_front_cm <= 20.0):
```

```
self.state = "STOP"  
return # Le jumeau numérique refuse d'avancer
```

Analyse Logique Python : Dès que `headless_simulator.py` reçoit la distance du HC-SR04, il met à jour `agv.distance_front_cm`. L'agent Python détecte l'obstacle et passe en état `STOP`. Cela crée une **Sécurité Redondante** : le robot physique s'arrête de lui-même, et le jumeau numérique confirme cet arrêt sur le Dashboard 3D, empêchant le système central de planifier d'autres mouvements tant que l'obstacle est présent.

2.3. Réception Série Non-Bloquante (Serial Polling)

Pour assurer une réactivité maximale du HC-SR04, la réception des commandes moteur via Serial a été transformée en boucle asynchrone (buffer caractère par caractère) au lieu de fonctions bloquantes.

```
while (Serial.available() > 0) {  
  char c = Serial.read();  
  if (c == '\n') {  
    sscanf(serialBuffer.c_str(), "%d,%d,%d,%d", &m1, &m2, &m3, &m4);  
    setMotor(1, m1); /* ... */  
    serialBuffer = "";  
  } else {  
    serialBuffer += c;  
  }  
}
```

Cela garantit que l'ESP32 peut continuer de lire les capteurs ultrasons et de maintenir la connexion MQTT active via `client.loop()`, sans jamais être interrompu par une latence de transmission série.

3. Optimisation de l'Architecture Frontend et du Trafic

L'affichage est désormais parfaitement intégré au Digital Twin en 3D.

Modifications de l'Architecture Frontend :

- **Architecture Headless** : Le jumeau numérique s'exécute à 100% via `headless_simulator.py`, dont le seul rôle est de calculer les mathématiques, les Mutex, les trajectoires (Dijkstra) et d'émettre les états via WebSocket.
- **Dashboard React / Three.js** : Tout l'affichage (véhicules, entrepôt, métriques, télémétrie en direct) est dorénavant géré par le portail Web (Frontend React) pour une expérience hautement immersive et performante.
- **Traçabilité de la Zone X** : L'agent Python notifie désormais explicitement ses demandes d'accès à la zone critique (`[TRAFFIC]` 🚦 AGV-01 a obtenu l'accès `exclusif`), offrant une clarté totale pour le débogage, tandis que le Dashboard 3D affiche une alerte visuelle rouge.

4. Sécurité Industrielle et Réduction de Latence (Failsafes)

Pour faire face aux pannes potentielles du réseau (Wi-Fi) ou aux crashes du serveur Python, le robot a été transformé pour fonctionner de manière autonome en mode de survie (Failsafe).

4.1. Watchdog Timer des Commandes

Si le serveur Python cesse d'envoyer des commandes de vitesse (Keep-alive) pendant plus de **1.5 secondes** (1500 ms), le microcontrôleur déclenche une sécurité Watchdog et force tous les moteurs à 0. Cela empêche le robot de continuer sa course "à l'aveugle" en cas de crash du serveur.

4.2. Arrêt d'Urgence sur Perte de Connexion MQTT

La fonction `client.connect()` étant bloquante, le code C++ a été modifié pour forcer un arrêt complet des moteurs **avant** d'entamer une quelconque tentative de reconnexion au réseau, garantissant ainsi l'immobilité totale pendant les périodes d'indisponibilité du Wi-Fi.

4.3. Élimination du Goulot d'Étranglement (Bottleneck) de Télémétrie

Auparavant, la télémétrie n'était envoyée que toutes les 2 secondes, causant un décalage visuel important dans le Dashboard. L'intervalle `TELEMETRY_INTERVAL` a été abaissé à **200 ms**, offrant un rafraîchissement 5 fois par seconde et garantissant une synchronisation Real-Time parfaite entre le mouvement physique et le jumeau numérique 3D.

5. Diagramme de la Séquence d'Arrêt d'Urgence

Voici l'illustration du flux de contrôle de l'arrêt d'urgence matériel intégré à l'ESP32 :

